UNIVERSIDAD
Panamericana

PROGRAMACIÓN AVANZADA

Sesión 13

Torres de Hanoi: Solución Recursiva

✎ NOTAS DEL PROFESOR

Universidad Panamericana
Dr. Miguel Alcaraz Rivera • malcaraz@up.edu.mx

2026

Índice

1 Estructura de la sesión **2**

2 Cómo conducir el concurso de votación **2**
2.1 Logística 2

3 Presentación del algoritmo recursivo **3**
3.1 Cómo presentarlo en el pizarrón 3
3.2 El algoritmo 3

4 El árbol de recursión — cómo explicarlo **4**

5 FAQs de los alumnos **4**

6 Soluciones a los ejercicios **6**

7 Solución al reto extra — Fibonacci y la razón áurea **7**

8 Cierre del arco de tres sesiones **8**

9 Resumen para el profesor **9**

Estructura de la sesión

Tiempo	Bloque	Tipo
15 min	Concurso de diseño: presentaciones + votación	Grupal
5 min	Proyectar el código ganador; análisis rápido	Exposición
10 min	Presentación del algoritmo recursivo	Pizarrón
40 min	Actividad individual: integrar la función recursiva	Trabajo
5 min	Cierre: ejecutar el programa, contar movimientos	Demo
5 min	Reto extra (Fibonacci / razón áurea) para quien termina antes	Opcional
80 min		

El impacto de esta sesión depende del contraste con la sesión 12. Los alumnos pasaron 50 minutos moviendo discos a mano; hoy verán que **seis líneas de código** hacen exactamente lo mismo. Ese contraste entre esfuerzo humano y elegancia del algoritmo recursivo es el momento de la sesión. No lo adelantes: espera a que el código esté en pantalla y la sala vea la ejecución en vivo.

Cómo conducir el concurso de votación

- **Logística**

1. Antes de que lleguen los alumnos, asegúrate de tener acceso a los archivos `.cpp` subidos a Canvas. Si algún equipo no subió, acepta una foto del código en pantalla como evidencia suficiente.
2. Cada equipo tiene **30–60 segundos** para mostrar su programa en pantalla (o en proyector). No hace falta que expliquen el código; solo que demuestren que funciona en al menos dos estados distintos.
3. Votación: cada persona escribe en un papel tres números de equipo (el suyo propio no cuenta). Recoger los papeles o usar una votación de mano alzada por ronda.
4. En caso de empate, el desempate es: el equipo cuyo código sea *más fácil de entender en 30 segundos*. Tú decides.

El código ganador puede tener el estado de las torres en variables globales, en un struct, con arreglos en lugar de vectores, o con cualquier otra estructura. No importa: el objetivo es que el equipo ganador *explique* su diseño al grupo en 2–3 minutos, y luego entre todos integren la función recursiva.

Si el código tiene un problema estructural que haría imposible integrar la función recursiva (e.g., no hay forma de saber qué disco está en el tope de cada torre), tómalo como oportunidad didáctica: habla con el grupo de por qué la estructura de datos importa para la extensibilidad. Luego usa el segundo lugar o

el código base del cuaderno de la sesión 12.

Presentación del algoritmo recursivo

• Cómo presentarlo en el pizarrón

No des el código de inmediato. Primero haz las tres preguntas siguientes y escribe las respuestas en el pizarrón:

1. **"Para mover n discos de A a C, ¿cuál es el último disco que debes mover y cuándo?"**
   Respuesta: el disco n (el más grande), de A a C, y *solo puedes moverlo cuando los otros $n-1$ ya están en B*.

2. **"Para poner los $n-1$ discos en B antes de mover el grande, ¿qué problema tienes que resolver?"**
   Respuesta: mover $n-1$ discos de A a B usando C como auxiliar. Mismo problema, $n-1$ discos.

3. **"¿Y después de mover el disco grande?"**
   Respuesta: mover $n-1$ discos de B a C usando A como auxiliar. De nuevo el mismo problema.

Cuando los alumnos lleguen solos a "el mismo problema con $n-1$", escribe el código en el pizarrón y deja que el silencio haga su trabajo.

• El algoritmo

La función completa:

```
void hanoi(int n, char origen, char destino, char aux,
           vector<int>& torA, vector<int>& torB, vector<int>& torC) {
    if (n == 0) return;
    hanoi(n-1, origen, aux, destino, torA, torB, torC);
    moverDisco(origen, destino, torA, torB, torC);
    hanoi(n-1, aux, destino, origen, torA, torB, torC);
}
```

Si el código base usa nombres distintos de funciones o estructuras de datos diferentes, el esquema es el mismo: el caso base devuelve inmediatamente, y el caso general son dos sub-llamadas con el origen y destino permutados.

Si prefieres mostrar primero la versión mínima (solo imprime movimientos) y luego migrar al código base:

```
void hanoi(int n, char origen, char destino, char aux) {
    if (n == 0) return;
```

```
    hanoi(n-1, origen, aux, destino);
    cout << "Mueve disco de " << origen << " a " << destino << endl;
    hanoi(n-1, aux, destino, origen);
}
```

Esta versión es útil para demostrar el algoritmo en el pizarrón sin depender del código base del concurso. Después los alumnos reemplazan el `cout` por la llamada a `moverDisco` con las torres reales.

El árbol de recursión — cómo explicarlo

Dibujar el árbol en el pizarrón es más efectivo que explicarlo de palabra. Hazlo para $n = 2$ primero (3 movimientos, árbol pequeño):

```
        hanoi(2, A, C, B)
        /       |       \
  hanoi(1,A,B,C) A→C  hanoi(1,B,C,A)
   /    |    \             /    |    \
h(0) A→B   h(0)         h(0) B→C   h(0)
```

Señalar:

- Cada hoja es un caso base (`n=0`, no hace nada).
- Cada nodo intermedio hace un movimiento (`moverDisco`).
- Para $n = 2$: 3 nodos intermedios = 3 movimientos = $2^2 - 1$.

Luego preguntar: "¿cuántos nodos intermedios hay para $n = 3$?" Los alumnos deben llegar a $2 \times 3 + 1 = 7 = 2^3 - 1$.

Esta es la recurrencia exacta que vieron en la sesión 12: $M(n) = 2\,M(n-1) + 1$. Hoy la ven en el árbol de llamadas: cada nodo de nivel k genera dos nodos de nivel $k + 1$ más un movimiento. Es la misma ecuación. Subrayarlo: el código y la matemática dicen exactamente lo mismo.

FAQs de los alumnos

Pregunta: Me confundo con qué torre poner como auxiliar en cada sub-llamada.
Respuesta: Hay un truco para no confundirse. Si quieres mover $n - 1$ discos de *origen* a *aux*, entonces *destino* queda libre y se convierte en el nuevo auxiliar. Y viceversa. Resumido: lo que era `destino` pasa a ser `aux` en la primera sub-llamada; lo que era `origen` pasa a ser `aux` en la segunda. El patrón es:

	origen	**destino**
Primera sub-llamada	`origen`	`aux`
Movimiento principal	`origen`	`destino`
Segunda sub-llamada	`aux`	`destino`

Pregunta: ¿No sería más natural detenerse cuando queda 1 disco?
Respuesta: Ambas versiones funcionan. Con $n = 0$ el código es más simétrico: cuando ya no hay discos que mover, simplemente regresa. Con $n = 1$ como caso base, habría que escribir el movimiento de ese disco explícitamente. La versión con $n = 0$ deja que el caso recursivo maneje incluso el disco unitario, lo que hace el código más corto. En la práctica, muchos textos usan $n = 1$; ninguna es incorrecta.

Pregunta: Si el árbol tiene $2^n - 1$ nodos internos, ¿cuántos nodos totales tiene (incluyendo hojas)?
Respuesta: Un árbol binario completo de profundidad n tiene 2^n hojas y $2^n - 1$ nodos internos, en total $2^{n+1} - 1$ nodos. Para $n = 3$: $2^4 - 1 = 15$ llamadas totales (8 de ellas con $n = 0$, que regresan inmediatamente).

Pregunta: ¿Existe una versión iterativa de Hanoi?
Respuesta: Sí, pero no es trivial. Hay dos enfoques:

1. **Simulación de stack:** usar un `stack` explícito que imite lo que el call stack haría de forma automática. Técnicamente es iterativo, pero conceptualmente es recursión disfrazada.
2. **Regla del bit menos significativo:** en el paso k, mover el disco cuyo número corresponde al bit menos significativo de k en binario. Esta regla (de la tabla de la sesión 12) genera la solución óptima sin recursión ni stack. Es elegante pero no intuitiva.

Para esta sesión no es necesario implementarlo.

Pregunta: Con $n = 25$ el programa tarda mucho. ¿Es normal?
Respuesta: Completamente normal. $2^{25} - 1 \approx 33$ millones de movimientos. Si además dibujas las torres en cada paso, la salida en terminal se convierte en el cuello de botella. La solución es no dibujar para n grande: solo contar los movimientos. La complejidad es $O(2^n)$ y no hay forma de mejorarlo para el problema estándar de tres agujas.

Soluciones a los ejercicios

`hanoi(2, A, C, B)` genera:

1. `hanoi(1, A, B, C)` $\rightarrow$ mueve disco 1: A $\rightarrow$ B
2. `moverDisco(A, C)` $\rightarrow$ mueve disco 2: A $\rightarrow$ C
3. `hanoi(1, B, C, A)` $\rightarrow$ mueve disco 1: B $\rightarrow$ C

Orden de movimientos: disco 1 (A$\rightarrow$B), disco 2 (A$\rightarrow$C), disco 1 (B$\rightarrow$C). Total: 3 movimientos = $2^2 - 1$.

Nodos internos: $2^4 - 1 = 15$ (movimientos reales). Hojas (`n=0`): $2^4 = 16$. Total: $15 + 16 = 31 = 2^5 - 1$ llamadas a la función. En general para n discos: $2^{n+1} - 1$ llamadas totales.

Versión con traza similar a `sumaTraza` de la sesión 11:

```
void hanoi(int n, char origen, char destino, char aux,
           vector<int>& torA, vector<int>& torB, vector<int>& torC,
           int nivel = 0) {
    if (n == 0) return;
    string indent(nivel * 2, ' ');
    cout << indent << ">> mover " << n << " discos de "
         << origen << " a " << destino << endl;
    hanoi(n-1, origen, aux, destino, torA, torB, torC, nivel + 1);
    moverDisco(origen, destino, torA, torB, torC);
    cout << indent << "  [disco movido: " << origen
         << " → " << destino << "]" << endl;
    hanoi(n-1, aux, destino, origen, torA, torB, torC, nivel + 1);
}
```

Sí (ver FAQ 4 de esta sección). Una respuesta completa menciona: la versión con stack explícito existe y es correcta; la versión basada en el bit menos significativo es $O(2^n)$ en tiempo pero $O(1)$ en espacio adicional. Ambas son válidas.

Ejemplo de lógica en `main`:

```
int n;
cout << "¿Cuántos discos? ";
cin >> n;
```

```
if (n < 1 || n > 30) {
    cout << "Número de discos fuera de rango (1-30)." << endl;
    return 1;
}
// inicializar torres con n discos en A
// ...
bool mostrar = (n <= 20);
hanoi(n, 'A', 'C', 'B', torA, torB, torC, mostrar);
cout << "Movimientos: " << movimientos
     << " (esperado: " << (1LL << n) - 1 << ")" << endl;
```

El parámetro `mostrar` se pasa a la función y controla si llama a `dibujarTorres` después de cada movimiento.

Solución al reto extra — Fibonacci y la razón áurea

Implementación mínima correcta:

```
long long fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}

int main() {
    cout << fixed << setprecision(6);
    cout << " n     F(n)     F(n)/F(n-1)\n";
    for (int n = 2; n <= 20; n++) {
        long long fn  = fib(n);
        long long fn1 = fib(n - 1);
        cout << setw(2) << n
             << setw(9) << fn
             << "     " << (double)fn / fn1 << "\n";
    }
}
```

La razón converge a $\varphi = (1+\sqrt{5})/2 \approx 1.618034$. Para $n = 20$ ya tiene 6 cifras correctas.

El árbol de `fib(5)`:

```
                fib(5)
               /      \
          fib(4)       fib(3)
         /     \       /    \
     fib(3)  fib(2) fib(2) fib(1)
```

```
        /   \   /   \  /   \
     fib(2) f(1) f(1) f(0) f(1) f(0)
     /   \
  fib(1) fib(0)
```

Diferencia clave: `fib(3)` aparece *dos veces*, `fib(2)` aparece *tres veces*. Hanoi, en cambio, no repite ningún subproblema: cada par (n, origen, destino) es único.

Esta distinción (subproblemas superpuestos vs. subproblemas disjuntos) es el criterio que separa los problemas que se benefician de memoización o programación dinámica de los que no. Es un buen gancho para el tema siguiente si el programa lo cubre.

Número de llamadas a `fib(n)`: $2\,F(n+1)-1$ en la versión recursiva ingenua. Para $n = 20$: $2 \times 10946 - 1 = 21891$ llamadas, versus solo $2^{21} - 1 = 2097151$ para Hanoi — aquí Hanoi es *más lento*, lo que sorprende a los alumnos porque Fibonacci "se ve más difícil". Usar ese contraste en la discusión.

No explicar el reto al grupo: entregarlo impreso o proyectarlo y dejarlo trabajar en silencio. Quien termina la actividad principal con tiempo de sobra avanza solo.

Si al final hay tiempo de discusión, la pregunta que más reacción genera es: "¿Cuántas veces se llama `fib(20)`?" La mayoría esperará un número cercano a 20; la respuesta correcta ($21\,891$) los sorprende y abre la conversación sobre memoización.

Cierre del arco de tres sesiones

Al final de la sesión, vale la pena hacer una síntesis breve en pizarrón:

Sesión	Tema	Qué aprendieron
11	Funciones recursivas	Caso base, caso recursivo, call stack
12	Hanoi a mano + diseño	El problema "se siente" antes de resolverse
13	Algoritmo recursivo	6 líneas hacen lo que tomó 50 min a mano

"La recursión no es un truco del programador: es la forma natural de describir un problema que se repite a menor escala. Hanoi lo hace evidente porque la solución recursiva *es* la definición del problema, escrita en C++."

Resumen para el profesor

✔ Recuerda

- El concurso toma 15 minutos; tenerlo preparado antes de que lleguen los alumnos acelera el proceso.
- No dar el código antes de las tres preguntas de arranque.
- La actividad individual dura 45 minutos; circular para verificar que todos hayan integrado `moverDisco` (no solo impriman movimientos).
- Cerrar con la ejecución en vivo y el conteo de movimientos.
- Al final del arco de tres sesiones, conectar con el siguiente tema: las funciones recursivas son la base de algoritmos de ordenamiento (mergesort, quicksort) que verán más adelante.